

HW4 Discussion: Generative Models

Machine Learning 2 - 00970209

July 17, 2026

1 Model Architecture and Training Procedure

Architecture: The implemented model utilizes a Deep Convolutional GAN (DCGAN) architecture. The Generator maps a 100-dimensional latent vector z into a 64×64 RGB image using ConvTranspose2d layers, accompanied by BatchNorm2d and ReLU activations. Conversely, the Discriminator employs Conv2d layers, BatchNorm2d, and LeakyReLU activations to downsample and classify the input images as real or fake.

Optimization & Hyperparameters: The networks were optimized using the Adam optimizer with a learning rate of $\alpha = 0.0002$ and a momentum parameter $\beta_1 = 0.5$. These specific hyperparameters are standard for DCGANs as they help stabilize the adversarial training dynamics and mitigate severe oscillations. The model was trained for 50 epochs with a batch size of 128. The model was highly sensitive to these hyperparameter choices; early experiments with different learning rates or momentum values often resulted in immediate training instability, highlighting the challenge of balancing the minimax game.

Training Dynamics & Instabilities: The training logs demonstrate significant fluctuations, characteristic of GAN training. Early in the process (e.g., epoch 0), the Generator loss spikes dramatically (reaching up to 24.88) while the Discriminator loss drops almost to zero. This indicates transient periods where the Discriminator easily overpowered the Generator, resulting in vanishing gradients. Furthermore, we observed clear training instabilities in the form of partial mode collapse (discussed further in Section 4), where the generator collapsed to producing a limited variety of flower shapes to reliably fool the discriminator.

2 Training Loss Plots and Convergence

Unlike supervised learning models where the loss is expected to monotonically approach zero, GAN training involves a minimax two-player game. Therefore, the loss plots do not smoothly converge to zero. Instead, the logs show continuous fluctuations for both the Discriminator loss (LD) and Generator loss (LG).

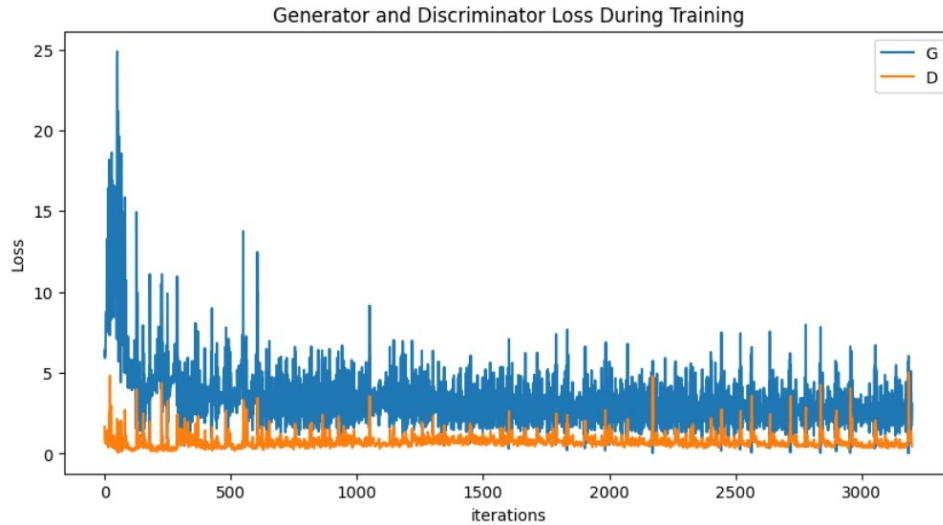


Figure 1: Generator and Discriminator loss over training iterations.

As seen in Figure 1, after the initial severe spikes, the two losses begin to bounce around a wider but roughly stable band. Because neither loss diverges to infinity or drops permanently to zero, the adversarial game reaches a persistent back-and-forth equilibrium, though, as Section 4 shows, numerical stability alone does not guarantee healthy sample diversity.

3 Summarizing Conclusions

Based on the experimental results, we can conclude that the DCGAN architecture is capable of successfully mapping random noise distributions to the high-dimensional image space. However, the training process is highly sensitive to the dynamic balance between the Generator and Discriminator. The recorded loss fluctuations emphasize the brittle nature of the adversarial minimax objective.

4 Generated Images Quality



Figure 2: Generated 64x64 flower images sampled from the latent space.

As seen in Figure 2, the model successfully outputs 64×64 images, and the basic color palettes and vague shapes of flowers are recognizable (e.g., pink and purple central structures with dark or green backgrounds). However, the generated samples exhibit clear visual artifacts and a noticeable lack of detail.

More importantly, the results show strong evidence of partial mode collapse. Despite being sampled from different random latent vectors z , the generated images lack meaningful diversity. The vast majority of

the outputs converge on very similar pinkish-purple structures, failing to capture the full variance and distinct classes expected from a complete flower dataset. This indicates that the Generator learned to exploit a limited set of outputs that successfully fool the Discriminator, rather than mapping the entire latent space to diverse image representations.

5 Latent Space Visualization



Figure 3: A pool of generated images used for latent space analysis and pair selection.

To evaluate the meaningfulness of the learned representations, we generated a large pool of images (Figure 3) and selected 3 pairs of similar images alongside 3 pairs of dissimilar images to analyze the L2 norm distances between their latent vectors z .

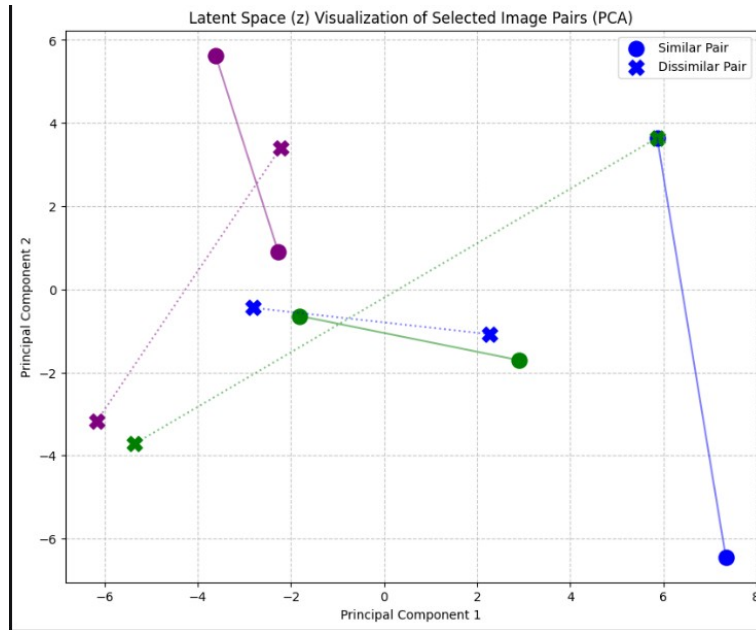


Figure 4: Latent space representation highlighting the original z vectors of the 3 similar pairs and 3 dissimilar pairs.

In a well-behaved and structured latent space, vectors that are mathematically close (low L2 distance) should decode into visually similar images. Conversely, visually dissimilar output images should correspond to latent vectors that are situated further apart in the space.

```

--- Calculating L2 Distances for Chosen Pairs ---
L2 Distance between vectors of images 26 and 27 is: 13.0474
L2 Distance between vectors of images 61 and 62 is: 14.4354
L2 Distance between vectors of images 2 and 40 is: 12.6307
L2 Distance between vectors of images 7 and 48 is: 14.2436
L2 Distance between vectors of images 26 and 34 is: 15.4510
L2 Distance between vectors of images 63 and 60 is: 15.0483

```

Figure 5: Calculated L2 norm distances between the selected image pairs.

Based on the calculated distances between the 3 chosen pairs of similar images and 3 pairs of dissimilar images, the similar pairs range from approximately 12.63 to 14.44 (mean ≈ 13.37), while the dissimilar pairs range from approximately 14.24 to 15.45 (mean ≈ 14.91). The similar pairs are, on average, closer together in latent space than the dissimilar pairs, and the two groups are cleanly separated in Figure 4's PCA projection. However, the numeric ranges still overlap slightly — the largest similar-pair distance (14.44, images 61-62) exceeds the smallest dissimilar-pair distance (14.24, images 7-48). This indicates that while the Generator has begun to learn a latent structure where proximity correlates with visual similarity on average, the latent space manifold is not yet sharply or reliably separated: L2 distance alone is only a moderate predictor of perceptual similarity, consistent with the partial mode collapse discussed in Section 4.

6 Note on Class-Conditioned Generation

The architecture implemented in this submission is an unconditional DCGAN: the Generator's only input is a random latent vector z , with no class label fed into either the Generator or the Discriminator at any

stage of training. This design choice follows the assignment's architectural note that the generator need only map a latent vector z to an output image, and it is consistent with the visualization tasks in Sections 4 and 5, which only require a diverse, non-single-class pool of generated samples rather than class-targeted generation.

One consequence of this design is relevant to the `reproduce_hw4()` function in `hw4_generation.py`. Since the Generator has no mechanism to condition its output on a specific flower category, `reproduce_hw4()` samples $\text{num_classes} \times \text{images_per_class} = 1,020$ images from independent random latent vectors z , rather than generating exactly 10 images belonging to each of the 102 known classes. The returned batch is therefore a random, class-agnostic sample of the model's output distribution and is not internally organized or labeled by class.

Producing genuinely class-targeted samples would require reformulating the model as a conditional GAN (e.g., a class embedding concatenated with z at the Generator's input, and the corresponding label fed to the Discriminator, as in a conditional DCGAN or ACGAN) and retraining from scratch with that conditioning signal. Given the time and compute constraints of this assignment, we opted to keep the unconditional architecture described above and to document this limitation explicitly here rather than misrepresent the outputs of `reproduce_hw4()` as class-controlled.